hashlock.

2024

Penetration Testing

ewe technology (Wallet)

December 2024

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The ewe technology/N,B,A team partnered with Hashlock to conduct a security Penetration Test of their Benty Mobile Application and API. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

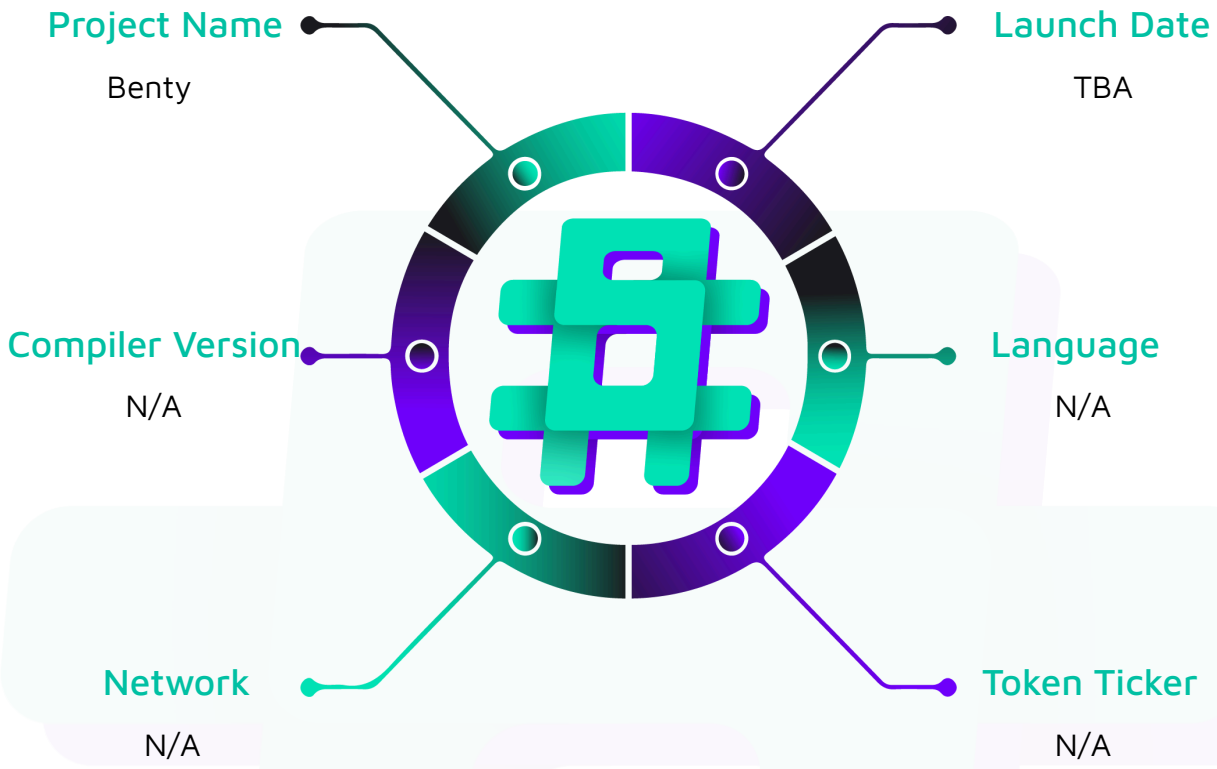
Benty is a mobile application that serves as a cryptocurrency wallet. It allows sending, receiving, and swapping and has a built-in cryptocurrency bridge functionality between multiple blockchains. Additionally, it is possible to stake your funds and watch transactions directly from the app. ewe technology/N,B,A is a decentralized finance (DeFi) Irevolution that is developing cutting-edge decentralized applications (dApps) and blockchain solutions. Our mission is to build secure, scalable, and innovative platforms that empower users to take control of their digital assets and participate in a decentralized financial ecosystem. With a focus on user experience and blockchain technology.

Project Name: Benty wallet
Website: https://ewetechnology.com/
Logo:

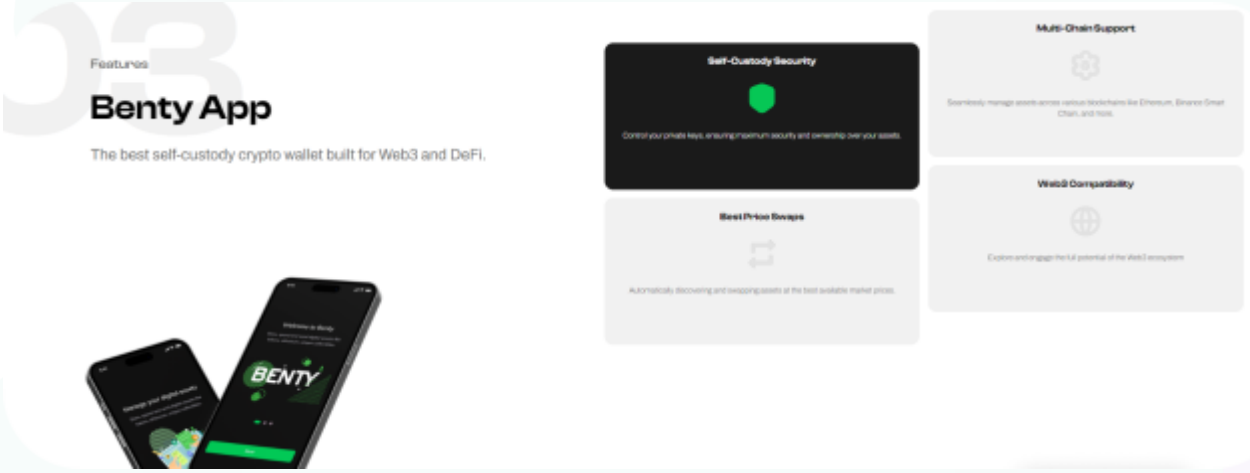
ewe technology

Next Blockchain Applications Limited

Visualised Context:



Project Visuals:



Penetration Test scope

At Hashlock, we executed a comprehensive penetration test on the Benty project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	**Benty Mobile Application and API**
Platform	**Android / iOS**
Penetration Test Date	**December, 2024**
Android application	benty-rc-v1.0.0-2.apk
APK MD5 hash	3b8a57849d30f32843708474f0f2b26c
iOS application	Benty Wallet v1.0.0 shared through the TestFlight
API spec	Benty-API.postman_collection.json
Postman collection MD5 hash	522ae908ef9d169b778d4e12d5520615
API URL	http://asia-northeast1-ewe-remixdao-prod.cloudfunctions.net

Security Rating

After Hashlock's Penetration Test, we found the Mobile Application and API to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Penetration Test Findings section.

We initially identified some significant vulnerabilities that have since been addressed.

Hashlock found:

3 Medium-severity vulnerabilities

3 Low-severity vulnerabilities

2 QA vulnerabilities

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Code Quality

This Penetration Test scope involves the Android, iOS Mobile Applications and related API, as outlined in the Penetration Test Scope section. The entirety of the project and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

Penetration Test Resources

We were given the Benty projects in the form of Android APK, iOS TestFlight access and Postman collection for the API testing.

The applications include a cryptocurrency wallet, with identical functionality and appearance on both platforms. Additionally, the Postman collection includes all the backend logic responsible for managing the investment account and acquiring data from on-chain and external providers.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Penetration Test Findings

Medium

[M-01] API - Missing rate limiting implementation

Description

The absence of rate limiting on API endpoints makes the system vulnerable to brute force attacks, DoS attempts, and API abuse.

Vulnerability Details

It was found that the whole API tested, that is http://asia-northeast1-ewe-remixdao-prod.cloudfunctions.net, does not enforce any kind of protection against brute force and flooding attacks. This could lead to service degradation and increased server costs due to excessive requests.

This vulnerability spans across all API endpoints, including sensitive areas such as authentication and transaction-related functionalities.

Impact

The lack of rate limiting exposes the system to multiple serious security risks. The API is susceptible to Denial of Service (DoS) attacks that could overwhelm server resources, leading to service degradation or complete unavailability for legitimate users. This vulnerability also enables potential data harvesting through rapid API calls and allows for automated scanning and enumeration attacks. The business impact includes increased operational costs due to excessive server load and potential service disruptions affecting user satisfaction and platform reliability.

Recommendation

A comprehensive rate limiting solution should be implemented across all API endpoints. The system should employ a combination of token bucket algorithm for flexible rate limiting and sliding window counting for accurate request tracking. Rate limits should be

configured based on both IP address and user account/API key, with different thresholds for various endpoint types. Authentication endpoints should be limited to 5-10 requests per minute, while general API endpoints could allow 60-100 requests per minute. The system should return proper 429 (Too Many Requests) responses including Retry-After headers and clear error messages

Status

Resolved

[M-02] Android - Insecure Android backup implementation

Description

The mobile application currently utilizes Android's default backup functionality without proper security controls or data filtering.

Vulnerability Details

Through analysis, it was determined that sensitive wallet data, including private keys, security credentials, and user preferences, are potentially included in the backup data set. The application's manifest file includes allowBackup="true" without any specific exclusion patterns or custom backup rules. This implementation relies on the standard Android Auto Backup feature, which automatically backs up application data to Google Drive when enabled by the user, without additional encryption beyond the platform's default mechanisms.

Impact

The improper implementation of backup functionality creates a serious security risk that could compromise user funds and private information. When backups are created, sensitive cryptographic material may be stored in less secure cloud storage environments, potentially exposing users to unauthorized access. If an attacker gains access to a user's Google account or backup files, they could potentially extract private keys and gain complete control over associated cryptocurrency wallets. The risk is amplified in situations where users restore backups to new devices or when sharing devices with multiple users.

Recommendation

A comprehensive secure backup solution should be implemented. The application should explicitly disable automatic backup for sensitive data by setting allowBackup="false" in the AndroidManifest.xml. A custom backup solution should be developed that includes several security layers:

- strong encryption of sensitive data before backup creation,
- secure key derivation from user credentials,
- clear separation of sensitive and non-sensitive data in backup files.

The backup process should implement validation checks to ensure no unencrypted sensitive data is included. Users should be clearly informed about what data is included in backups and provided with secure alternatives for backing up wallet information, such as manual encrypted exports.

Status

Resolved

[M-03] Android/iOS - Screenshot protection is not implemented

Description

The mobile application currently lacks any protection against screen capture functionality, particularly during the display of sensitive information such as private keys. Through testing, it was confirmed that users can freely capture screenshots of any screen within the application, including those containing critical security information. This vulnerability exists across both Android and iOS versions of the application, with no differential implementation of security measures between platforms.

These screenshots can potentially be accessed by other applications, backed up to cloud services, or inadvertently shared. In cases where private keys or seed phrases are captured, this could lead to complete compromise of user accounts

Recommendation

The application should implement comprehensive screenshot prevention mechanisms using FLAG_SECURE for Android activities and isSecureTextEntry for iOS screens where sensitive information is displayed. This protection should specifically target screens showing private keys, seed phrases, transaction signing interfaces, and account details. Additionally, the system should implement visual warnings when users enter screens with sensitive information and provide secure alternatives for necessary information sharing, such as structured export functions with proper encryption.

Status

Resolved

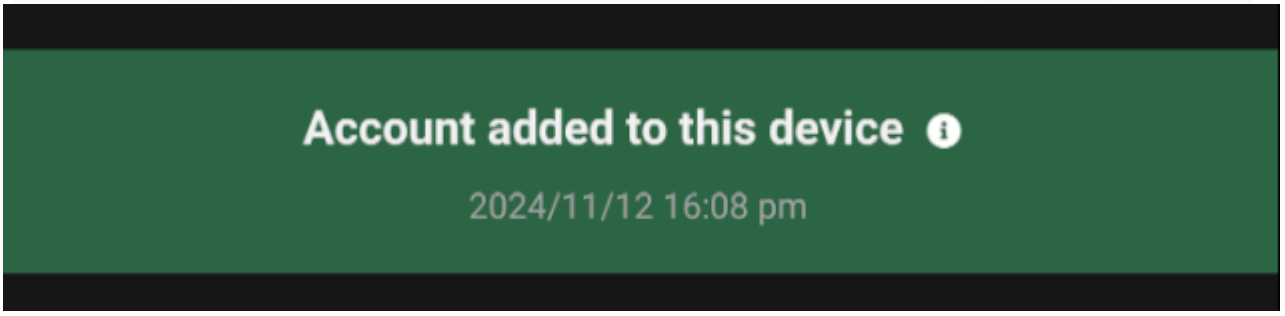
Low

[L-01] Android - Unexpected application logout in “Transaction” tab

Description

When a user clicks an “i” sign in the Transaction tab near to “Account added to this device” information, mobile application triggers an unintended logout mechanism that terminates the current active session.

This is unexpected, because this functionality does not have any kind of “logout” functionality near that specific button.



Such an application behaviour might be confusing and annoying for the users of the solution. The issue is only related to the Android version of the application, because the iOS version works properly in this area.

Recommendation

We suggest analyzing why the application logs the user out when clicking in the place described above, and then improving the code and/or graphical interface so that this does not happen.

Status

Resolved

[L-02] Android - Unnecessary audio recording permission

Description

The application requests and maintains the audio recording permission on Android (android.permission.RECORD_AUDIO) despite having no features or functionality that utilize audio capture capabilities. This permission is included in the Android Manifest file and is requested during the application's installation or first run.

Users may question the application's privacy practices when they notice audio recording capabilities in an application that shouldn't need them. This permission increases the application's attack surface unnecessarily, as any vulnerability in the application could potentially be exploited to abuse this permission. It may also lead to decreased user trust and increased likelihood of permission denial for other necessary features.

Recommendation

The audio recording permissions should be completely removed from the Android manifest file. Only functionally justified permissions should be granted.

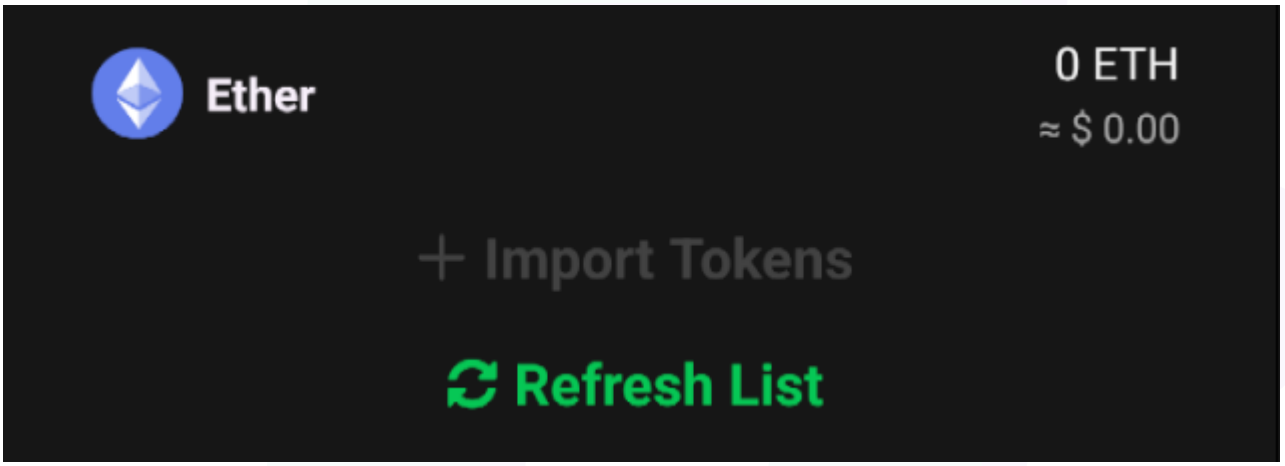
Status

Resolved

[L-03] Android/iOS - Non-functional token import feature

Description

The token import functionality, a core feature for any cryptocurrency wallet, is currently non-operational. Users attempting to add custom tokens through the provided interface encounter greyed-out button, which cannot be clicked or used. This functionality breakdown appears to be consistent across different app versions and platforms, suggesting a fundamental implementation issue rather than a temporary malfunction.



The non-functioning token import feature severely limits the wallet's utility and user experience. Users are restricted to pre-configured tokens, preventing them from interacting with new or custom tokens on their chosen blockchain networks.

Recommendation

Implement comprehensive input validation and sanitization for wallet names. Create a strict allowlist of permitted characters, limiting input to alphanumeric characters and basic punctuation that serves legitimate naming purposes.

Status

Resolved

QA

[Q-01] API - JWT structure inconsistency

Description

The JWT (JSON Web Token) structure undergoes unexpected changes during the token renewal process. Testing reveals that renewed tokens contain different payload structures, compared to the original tokens.

While no security issues have been discovered related to this issue, this implementation deviates from standard versions of JWTs that are renewed via the "Renew Token" mechanism and may lead to future security issues if some fields in the refreshed token are not available for validation.

Recommendation

A standardized JWT structure should be implemented across all token generation processes, including both initial creation and renewal.

Status

Acknowledged

[Q-02] Android/iOS - Insufficient input sanitization for wallet names

Description

The application allows users to create wallet names using any string input without proper sanitization or validation of special characters. Testing reveals that wallet names can include potentially dangerous characters such as <, >, ", ', /, , and other special characters commonly used in cross-site scripting (XSS) attacks.

While the current implementation might not immediately expose XSS vulnerabilities, the lack of input sanitization creates significant security risks for future development. If wallet names are ever displayed in a web context, embedded in HTML outputs, or parsed by JavaScript functions, they could potentially execute malicious scripts.

Recommendation

Implement comprehensive input validation and sanitization for wallet names. Create a strict allowlist of permitted characters, limiting input to alphanumeric characters and basic punctuation that serves legitimate naming purposes. All special characters can be converted to, for example, HTTP entities or URL encoded.

Status

Resolved

Conclusion

After Hashlocks analysis, the Benty project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au